

Algorithmization and Programming of Solutions

Part I

TOPIC #3

AN INTRODUCTION TO PROGRAMMING OF ALGORITHMS

RTU LECTURER OLGA YAKOVLEVA, MG.SC.ING

Table of contents

1. An introduction to programming of algorithms
 - basic approaches used in algorithmization
 - a programming language and a program
 - generations of programming languages
 - classifications of programming languages

1. An introduction to programming of algorithms

BASIC APPROACHES USED IN ALGORITHMIZATION, A PROGRAMMING LANGUAGE AND A PROGRAM, GENERATIONS AND CLASSIFICATIONS OF PROGRAMMING LANGUAGES

Basic approaches used in algorithmization (1/2)

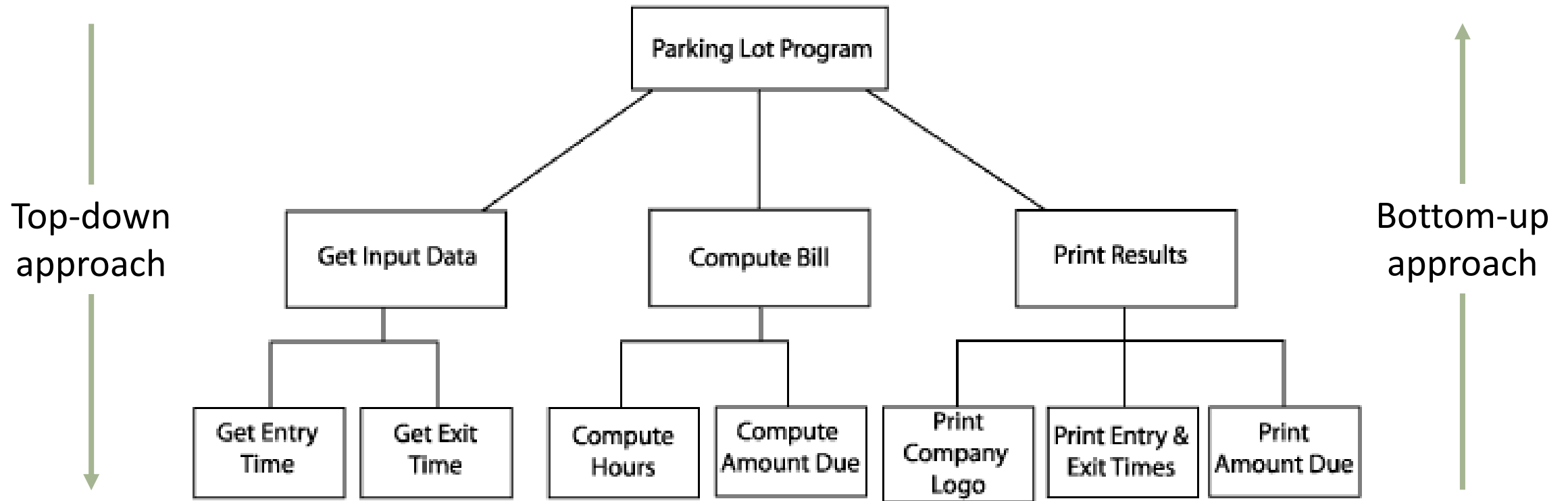
A top-down approach also referred to as **decomposition** is the process of dividing a task into parts (subtasks) to simplify algorithmization, programming and maintenance of a solution

If some subtask still looks complicated, it can be decomposed into smaller parts once again

A bottom-up approach is the process of linking existing or built-in parts of a solution together to create an integrated solution for a given task

Basic approaches used in algorithmization (2/2)

Original source of the picture: https://facweb.northseattle.edu/msteffancin/old_Matlab_EGR_141_site/day12chapter3.htm



A structure chart of a simple parking lot billing program

Using a top-down approach for creating an algorithm

New hotel in Riga needs online reservation software

The task: decompose the software into components by functions, create an algorithm for one of functions and represent it in a form of a flowchart

Guidelines:

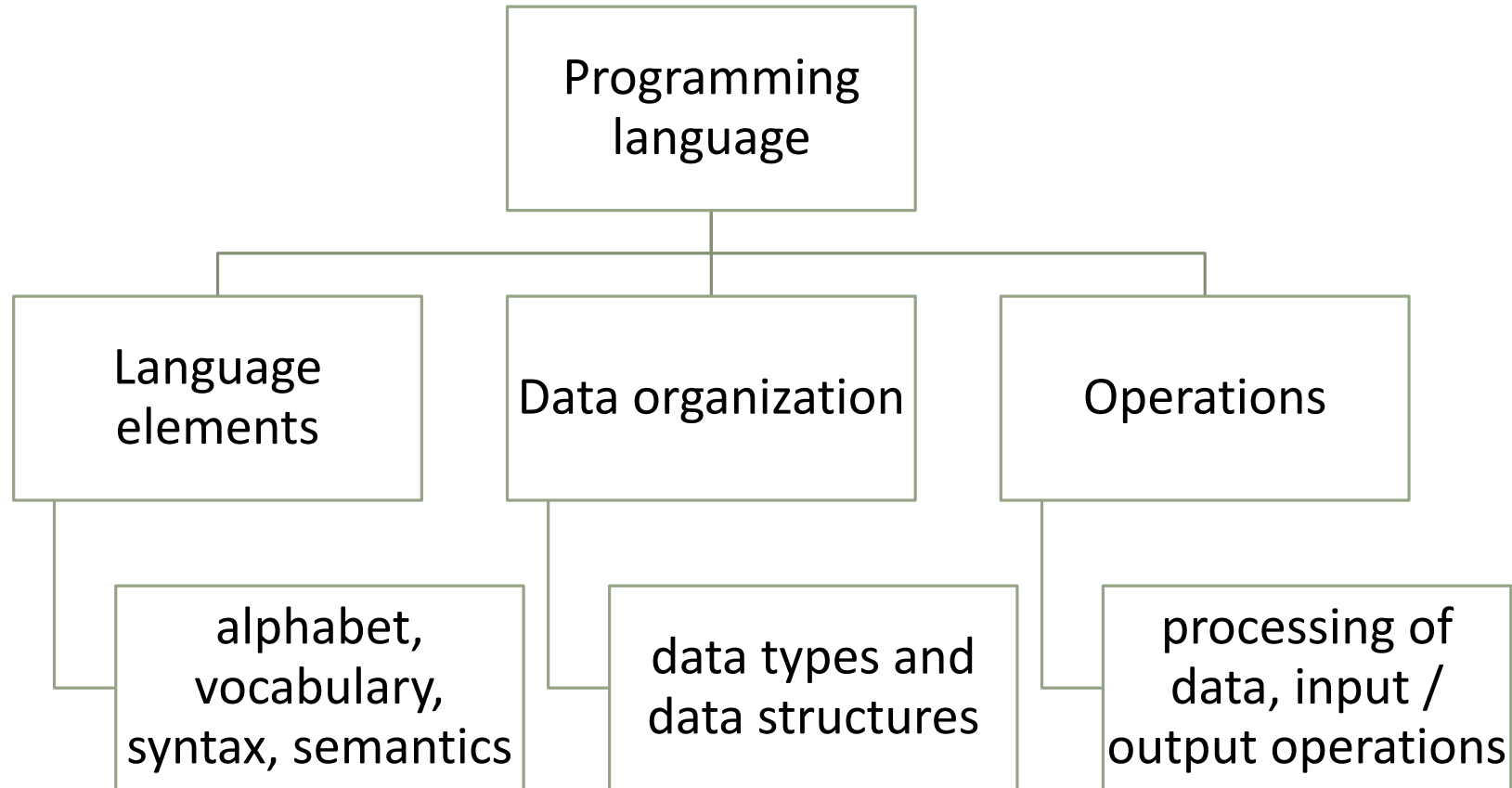
1. Define the main functions of online reservation software
2. Decompose the main functions into subfunctions (if needed)
3. Choose one function / subfunction and divide it into actions
4. Draw a flowchart

A programming language and a program

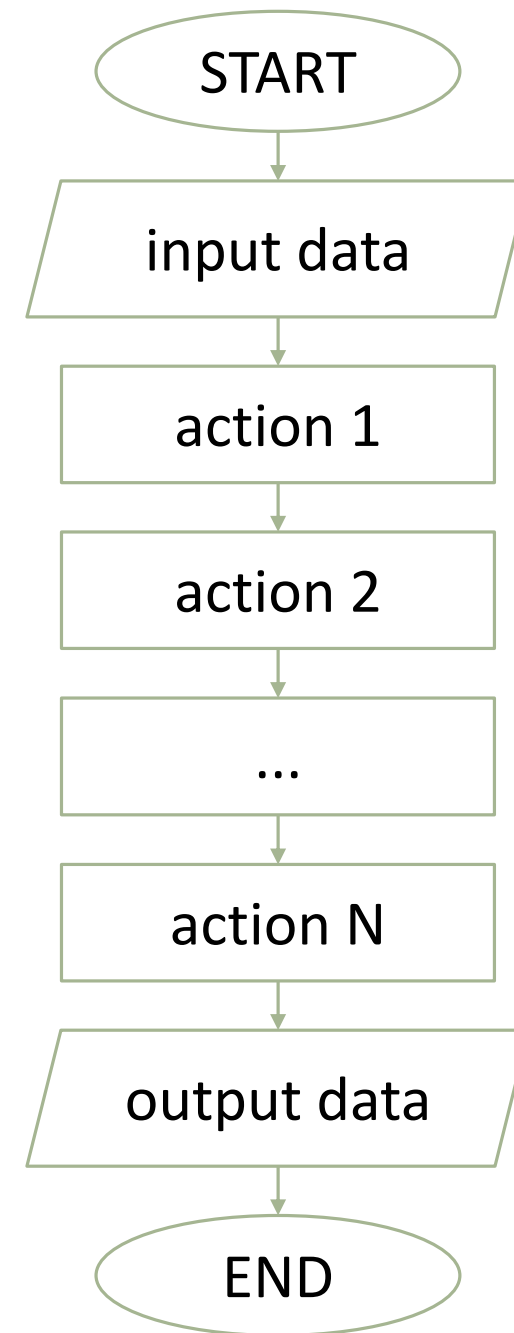
A programming language is a notation with a list of rules for representing algorithms that can be converted into machine readable code and executed by a computer

A program is an algorithm for a computer represented in some programming language, it includes a set of interrelated actions that should be executed to solve a given task

The organizational chart of a programming language



The most common
organizational
chart of a program



Generations of programming languages (1/2)

The first generation languages (1GL) are machine languages – a program is a set of instructions written in the octal or the hexadecimal number system that a computer's central processing unit can execute directly, e.g., *010D0E0A* (an addition operation in hexadecimal form)

The second generation languages (2GL) are assembly languages – a program is a set of a human-readable instructions in a machine language used to control specific computer operations, e.g. *ADD 2,3,4*

Generations of programming languages (2/2)

The third generation languages (3GL) are procedural languages – a program is a set of procedures that are called in a specified sequence, e.g., FORTRAN, Algol, Pascal, C, C++, Java, C#

The forth generation languages (4GL) are non-procedural languages – a program defines what to do, without specifying how to do it, e.g., SQL, HTML

Classifications of programming languages

Programming languages can be classified by:

- **the level of algorithm details**
- **generations**
- **programming paradigms**
- **intended domain of use**
- **purpose**
- etc.

Wikipedia article “List of programming languages” includes **694** notable programming languages in existence

Most popular programming languages according to IEEE Spectrum

Picture original source

https://spectrum.ieee.org/static/interactive-the-top-programming-languages-2019?utm_source=spectrum-hero

IEEE SPECTRUM				Topics ▾	Reports ▾	Blogs ▾	Multimedia ▾	Magazine ▾	Resources ▾	Search ▾
Language Ranking: IEEE Spectrum										
Rank	Language	Type			Score					
1	Python				100.0					
2	Java				96.3					
3	C				94.4					
4	C++				87.5					
5	R				81.5					
6	JavaScript				79.4					

Classification of PL by the level of algorithm details

Low-level language is a programming language that deals with a computer's hardware components and constraints; programs and applications written in low-level language are directly executable on the computing hardware without any interpretation or translation (Assembly)

High-level language is any programming language that enables development of a program in much simpler programming context and is generally independent of the computer's hardware architecture (C, C++, Java)

Classification of PL by programming paradigms (1/2)

A **programming paradigm** is a fundamental style of computer programming, serving as a way of building the structure and elements of computer programs

Imperative languages – specify explicit manipulation of the state of the computer system (SQL)

Procedural languages – specify an explicit sequences of steps to follow to produce a result (C)

Declarative languages – describe relationships between variables in terms of functions or inference rules (HTML)

Classification of PL by programming paradigms (2/2)

Functional languages – treat computation as the evaluation of mathematical functions (Lisp)

Object-oriented languages – implement objects and their associated procedures within the programming context to create programs (C++, Java)

Logic languages – based on formal logic (Prolog)

Classification of PL by intended domain of use

General-purpose languages – programming languages used to solve a wide variety of problems (C, C++, Java)

Domain-specific languages – programming languages designed to solve a specific class of problems:

- MATLAB is intended primarily for numerical computing, matrix manipulations, plotting of functions and data, implementation of algorithms
- HTML is designed to create web pages
- SQL is designed for managing data held in a relational database management system

Classification of PL by purpose

General purpose languages (C, C++, Java)

System programming languages (C, Simula, Haskell, Ruby)

Scripting languages (JavaScript, VBScript, PHP, Python, Ruby)

Domain-specific languages (MATLAB, HTML, SQL)

Concurrent/distributed languages – several computations are executing during overlapping time periods (Ada, C#)